

A New Algorithm For Robot Curve-Following Amidst Unknown Obstacles, And A Generalization Of Maze-Searching

A. Sankaranarayanan

Isao Masuda

Secom Intelligent Systems Lab
6-11-23, Shimorenjaku, Mitaka
Tokyo 181, Japan
e-mail: sankar@secom.co.jp

Abstract

A new, non-metric path planning algorithm *Curv1* is developed for moving a point automaton between two given points along a guide-track (i.e. a simple curve), amidst unknown obstacles. No physical mark is made on the guide-track and no position or distance information is used. The non-heuristic algorithm is shown to converge and an upper bound on the path length is derived. This non-metric formulation is shown to be related to, and in some sense, a generalization of the maze-searching problem. For the curve-following task, the new algorithm *Curv1* is shown to be a generalization of the maze-searching *Pledge* algorithm [7]. The robustness of the new algorithm makes it suitable for an industrial application of autonomous robot guide-track following.

1 Introduction

In this work, we consider the problem of planning a path for a robot in a planar scene filled with unknown obstacles of arbitrary shapes. This problem is referred to in the literature as, motion planning with uncertainty, real-time, sensor-based motion planning with incomplete information, etc.

An alternative formulation, The Piano Movers' Problem [1], assumes a complete knowledge of the robot's global environment. Using this global description, a path is generated through an one shot, off-line, graph search procedure. The complexity of computing a path is the central issue. The essential aspects of the unknown scene model, as distinct from the Piano Movers' problem, are as follows:

1. At any instant, the robot can acquire, through its sensors, only a description of its immediate surroundings. A global knowledge of the scene is not available.
2. The robot's motion is determined using only the partial scene description collected up to the current instant. That is, a path is created piece by piece as the robot moves and observes its surroundings.
3. The piecewise path construction does not guarantee a final convergence to the goal point.

The third aspect on reaching the goal point leads to a classification of the algorithms as heuristic and non-heuristic (or provable).

A heuristic algorithm cannot guarantee a path reaching the goal point. Under Khatib's 'artificial potential function' based obstacle avoidance [2], a mobile robot can get trapped in a local potential minimum. On the other hand, a non-heuristic algorithm guarantees a path to the goal point. That is, within a restricted domain of scenes (e.g. planar), it can be proved that the algorithm will always generate a successful path.

In this work, we consider only the non-heuristic path planning algorithms for unknown environments.

2 Background

2.1 Robot Path Planning

A pioneering work in the area of non-heuristic robot path planning algorithms for unknown environments has been the formulation and solution of the following problem by Lumelsky and Stepanov [3].

Problem P0: A point mobile automaton (MA) has to be moved from a point S to a point T in a planar scene filled with unknown obstacles of arbitrary shapes. The obstacle boundaries are simple closed curves of finite lengths. At any instant, MA knows only its coordinates (in some reference frame) and those of target T . MA can detect and follow an obstacle boundary, when it meets one. MA can store the coordinates of any number of points on its path, for future use.

Lumelsky and Stepanov [3] developed two non-heuristic algorithms, *Bug1* and *Bug2*, to solve problem $P0$. Sankaranarayanan and Vidyasagar developed a general algorithm *Alg2* [5] to solve $P0$, which is optimal in having the smallest worst case path length possible for problem $P0$.

Problem $P0$ assumes that the robot knows its coordinates and those of target T . All the existing algorithms for $P0$ make use of the geometric data, like, position and distance. This prompts a further classification of all the non-heuristic algorithms for unknown scenes.

Definition 1 Metric algorithms use the geometric information of position and/or distance to generate a path in an unknown scene.

All the other algorithms for unknown scenes can be called non-metric. Non-metric algorithms use only information like, physical marks on the scene, angles (with re-

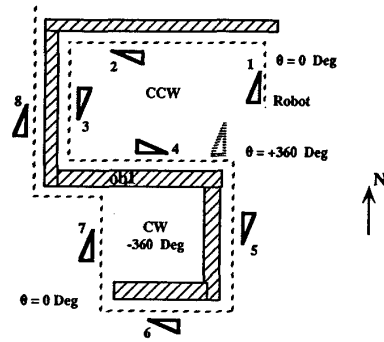


Figure 1: The measurement of the angle of turn in the Pledge Algorithm

spect to some reference direction), crossing of some known curve, etc.

In the present work, the non-metric algorithms are not allowed to make or use any physical marks on the scene.

2.2 Maze-Searching

Next, we consider the problem of escaping an unknown maze. The maze-searching differs from robot path planning in that there is no single goal point T to seek. Instead, the goal is to make the robot exit a certain region (e.g. a disc) containing the maze.

The Pledge algorithm [4, 7] is a non-heuristic procedure for a point automaton to escape an unknown maze. Interestingly, the algorithm does not make use of any position or distance information. Also, the point automaton does not make or use any physical mark on the scene. Instead, the robot uses a magnetic compass to obtain a fixed reference direction (e.g. north). Using this reference direction, the robot measures its angle of turn while following a wall in the maze.

The measurement of the angle of turn is explained through the example in Fig. 1. At location 1, the robot is moving along a straight line towards the north. Here, let the cumulative angle of turn θ , measured with respect to a north-south line, be zero. At location 4, the robot has completed a rotation in the counter-clockwise direction and θ is now $+360$ deg. At location 7, after a clockwise rotation, the angle θ becomes zero again.

The basic idea of the Pledge algorithm is that the robot can leave a wall and move straight towards the north, if and only if, the angle θ is zero. Hereafter, this condition will be referred to as, the angle of turn criterion. The Pledge algorithm is a non-metric algorithm as it does not use any position or distance information. A brief description of the algorithm along with an example in Fig. 2, is presented next.

The Pledge Algorithm:

Executing the following steps, the point automaton is guaranteed to cross an infinite straight line L , placed east-west, anywhere north of the maze.

1. Initialize a counter C to zero.
2. Move straight towards the north until an obstacle is met. Go to step 3.
3. On meeting an obstacle, turn left and follow the obstacle boundary. Increment counter C by $+1$ whenever the robot has made a $+360$ deg angle of turn

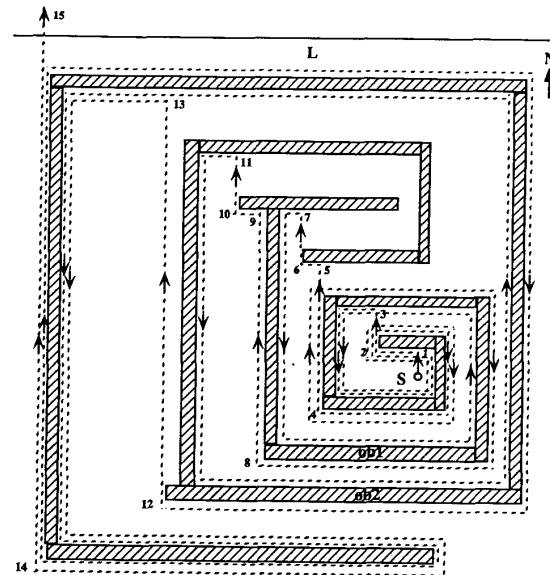


Figure 2: A point robot escapes a maze using the Pledge algorithm

(i.e. counter clockwise rotation), and -1 if the angle of turn is -360 deg.

4. Leave an obstacle if, (1) it is possible to move straight towards the north and (2) the counter C reads zero. Go to step 2.

In Fig. 2, MA makes several repeat trips along sections of the maze and eventually crosses the line L . The numbers 1, 2, ..., in the figure denote the sequence of leaving and meeting of the obstacles by MA. These numbers are shown only for an easy understanding of the figure and the algorithm does not use them. Lumelsky [4] presents an excellent review of the path planning algorithms for problem P0, and the Pledge algorithm for maze-searching.

2.3 The Problem Formulation

A non-metric algorithm will be more robust than a metric-one by not requiring any position or distance information. The present work has been inspired by the robust nature of the non-metric, maze-searching Pledge Algorithm. In this work, we aim to develop non-metric algorithms for robot path planning as well.

The path planning problem we analyze here is the robot navigation through guide track following. The robot detects and follows a guide track installed on the floor to go between the two given points. The guide-track following is a common industrial practice and will be discussed in more detail after the definition of a non-metric formulation of the robot guide-tracking problem. This problem forms the basis of the rest of this work.

Path Planning Problem P1: A point Mobile Automaton (MA) has to be moved in a 2-dimensional plane along a given simple (i.e. non self-intersecting) curve ST , from point S to point T . The plane is filled with unknown and arbitrarily shaped obstacles, with many of them lying

on curve ST . The obstacle boundaries are simple closed curves of finite lengths.

MA does not know its location (i.e. geometric coordinates) and cannot make any physical mark on either the obstacle boundaries or curve ST . MA can detect and follow an obstacle boundary or curve ST , when it meets one. While on curve ST , MA knows the direction to move along the curve to reach T . Clearly, without the coordinate measurements and physical marks, MA cannot decide if a given point has already been visited or not.

The layout of the rest of the paper is as follows: The various definitions and assumptions related to problem $P1$ are listed next. The motivation to study non-metric robot path planning algorithms, from both practical and theoretical points of view, is taken after that. The design of a new algorithm *Curv1* is presented next, followed by several examples of its operation. Various characteristics of the new algorithm, like, convergence, worst case path length, etc. are then derived. The relationship between the curve-following algorithm *Curv1* and the maze-searching *Pledge* algorithm is discussed next. Some future research directions are mentioned in the conclusion.

3 Definitions and Assumptions

The following definitions and assumptions follow from problem $P1$.

MA is a point object which does not know its position and therefore cannot store the coordinates of any point on its path. Also, MA cannot physically mark any point for a future reference. That means, MA cannot determine if it has already visited a given point on its path.

A guide-track is a non self-intersecting curve ST of length D . Hereafter, the terms, *curve ST* and *guide-track* will be used interchangeably. The curve ST is directed such that, at any point on ST , MA knows the direction to move along the curve to reach T . MA only knows that ST exists but does not know its actual description. MA starts at point S and moves towards T along ST .

All the obstacles in the scene are completely unknown. An obstacle ob_i 's boundary is a simple closed curve of finite perimeter p_i . For practicality, it is assumed that only a finite number of obstacles intersect curve ST . There is no restriction on the size and shape of the obstacles.

MA can detect and follow either an obstacle boundary or curve ST , when it meets one. Since all the obstacles are unknown, it is assumed, without loss of generality, that MA always turns left to follow an obstacle boundary, when it meets one.

MA meets an obstacle boundary, while moving along curve ST , at a *hit point*. Similarly it leaves an obstacle boundary, to follow curve ST , at a *leave point*.

When MA revisits any point on its path it is called a local cycle. In a non-trivial scene MA meets at least one obstacle on its way.

4 Robot Guide-Track Following

4.1 The Specifications

Autonomous robot navigation through *guide-track* following is a common industrial practice. Material transfer in a factory floor, transporting food and drugs in a large hospital, and monitoring and material transfer in a laboratory are some of the instances where guide-track following can be quite effective. Tsumura [6], describes various guide-track implementations, like, electromagnetic, optical reflecting tape, magnetic strips, laser, etc.

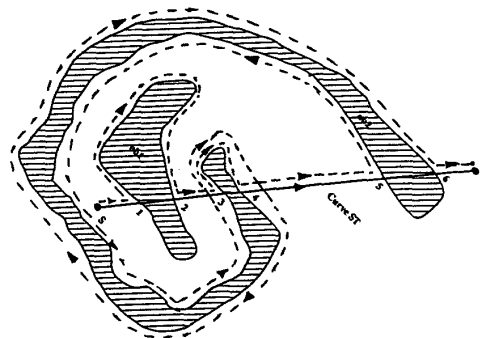


Figure 3: The *Pledge* algorithm applied to straight guide-track following

In a practical situation, unexpected objects (e.g. a box on a shop floor) are likely to lie on the guide-track. Since the motivation behind guide-track following is to minimize human assistance, it is essential to design a robust, autonomous method to deal with the unknown obstacles. Such a method has to be simple and require only an easily available sensory data. For example, the method should not need the exact coordinates of the robot (in some reference frame) as that requires elaborate land marks and detection systems at various points in the scene. That means, we need a non-metric procedure (i.e. not using any position or distance information) to deal with the unknown obstacles on the guide-track. Also, it is preferable not to let the robots make any physical marks on the guide-track as several robots may use the track at the same time.

4.2 The Pledge Algorithm For Guide-Track Following

The *Pledge* algorithm used for maze-searching does not need any position or distance information. It uses only the *angle of turn* measured with a magnetic compass. Also, the robot does not make or use any physical mark. If *Pledge* algorithm can be applied to guide-track following as well that would satisfy all the specifications listed above.

The scene in Fig. 3 has a straight guide-track ST . MA has to be moved from S to T along the line ST . The *Pledge* algorithm is applied to this scene with the following *minor* modifications.

1. The reference direction, measured by the compass, is set along the line segment ST , instead of along the north, as done earlier.
2. On satisfying the angle of turn criterion, MA leaves an obstacle and follows line ST , towards T .

The dotted line in Fig. 3 is the path due to the *Pledge* algorithm, with the arrows denoting the direction of MA 's travel. After traveling twice along a section of obstacle ob_2 's boundary, MA finally reaches T . As before, the hit and leave points are sequentially numbered to help follow the figure. It can be shown that the *Pledge* algorithm always works for all straight guide-track following.

The algorithm works for a straight track mainly because MA always moves in the reference direction after leaving an obstacle. This ensures a well defined measurement of the 'angle of turn'. But, for a curved guide-track, the angle of turn criterion becomes ill-defined. This is illustrated through the following example in Fig. 4. The curve ST is

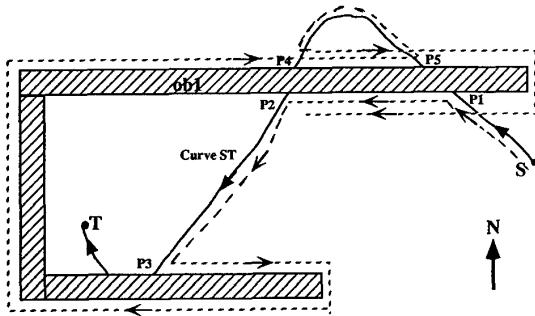


Figure 4: The *Pledge* algorithm fails for a curved guide-track

shown as a continuous line with the arrows denoting the direction from S to T .

Following the curve ST from S , MA meets an obstacle at $P1$ and starts following its boundary. When MA meets curve ST again at $P2$, it is not clear what the angle of turn is. It is close to zero with respect to the line $SP1$, but some large +ve value with respect to the tangent (assuming that it exists) to curve ST at $P2$. Since MA 's keeps changing its direction of motion while traveling along curve ST , let us assume a fixed direction, say, the north, to measure the 'angle of turn'. Then, since the angle of turn at $P2$ is zero, MA starts following the curve ST . For the same reason, it leaves the obstacle again at $P5$ and makes a counter-clockwise loop along curve ST to return to $P4$. Now the angle of turn is $+360$ deg. Proceeding from $P4$, MA returns to $P1$ with zero angle of turn. Since MA cannot determine if a point has already been visited or not, it will start following the curve ST again at $P2$. This puts MA in an infinite loop.

4.3 Some Theoretical Issues In Curve-Following

Since the *Pledge* algorithm does not work for general curve-following, we start the search for a new algorithm. At this juncture, the following questions arise:

1. Is there an algorithm to follow a general guide-track without using any position or distance information or physical marks?

It is not obvious that such an algorithm must exist.

2. If an algorithm is designed to work for all guide-tracks, how will its basic principle be related to those of the existing algorithms?

This is an enquiry into the general characteristics of a class of algorithms for unknown environments. The new algorithm will be under the subclass of non-metric algorithms. Up to now, *Pledge* algorithm is the only member of this subclass. *Pledge* algorithm is based on the fact that the angle of turn around a simple closed curve is ± 360 deg. The basic principle of the new algorithm may be either distinct or a generalization of the 'angle of turn' criterion.

3. How will the general algorithm perform as compared to the *Pledge* algorithm, for straight guide-tracks?

That is, we would like to know if the generality of the new algorithm will be obtained at the cost of producing more complex and longer paths, as compared to the *Pledge* algorithm, for the simple case of a straight line guide-track.

These questions shall be addressed as we proceed.

5 The Algorithm Curv1

In [4], Lumelsky points out that the path planning problem $P0$ can be thought of as a graph search problem. That is, the intersection points between the obstacle boundaries and the line ST form the nodes of an undirected, connected graph G which describes their interconnections. The start (S) and target (T) points are also nodes in this graph.

Reaching the target T in problem $P0$ can then be accomplished by searching this *unknown* graph G for the node T , starting from the node S . [4] describes two procedures, *Tarry's Algorithm* and *Fraenkel's Algorithm* to perform the graph search. These algorithms work by marking the nodes which have been visited.

These algorithms cannot be applied to the curve-following problem $P1$, as marking the visited nodes is *not* allowed. That is, the graph problem corresponding to the curve-following problem $P1$ is to search an unknown, partly directed (curve ST is directed) graph without marking any of the visited nodes.

The new algorithm is designed based on the topological property (from the Jordan Curve Theorem [8]) that the curve ST intersects an obstacle boundary at an even number of points (when both S and T are outside the obstacle boundary).

Next, we present the new algorithm *Curv1*. The algorithm can be followed through the examples in Figs. 6, 7 and 8. Fig. 5 illustrates how a counter C is set while executing the steps 3b (i), 3b (ii) and 3b (iii). Figs. 5a, 5b and 5c show a part of an obstacle boundary, an interval in curve ST and a part of MA 's path. The counter C 's readings, before and after crossing the curve ST , are also shown in these figures.

Procedure for Algorithm Curv1:

1. Set counter C to zero. Start from point S . Go to step 2.
2. Move along curve ST until one of the following occurs:
 - (a) The target T is reached. Stop.
 - (b) An obstacle is met. Follow the obstacle boundary in the local direction *left*. Go to step 3.
3. Follow the obstacle boundary until one of the following occurs:
 - (a) The target T is reached. Stop.
 - (b) The curve ST is met at a point P . One of the following steps is executed.
 - i. The counter C reads zero and, at P , MA can move along curve ST towards T . Leave the obstacle boundary and follow the curve ST (Refer to example in Fig. 5a). Go to step 2.

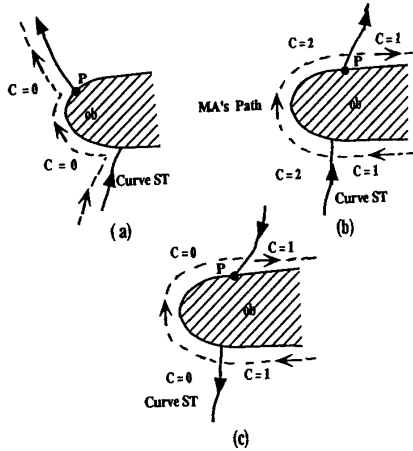


Figure 5: Counter C 's readings before and after crossing a point P

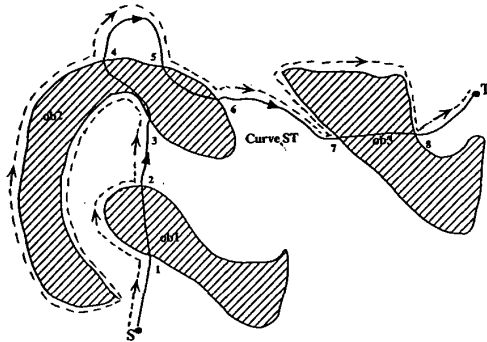


Figure 6: Algorithm *Curv1* on a simple scene and a simple curve ST

- ii. The counter C is non-zero and, at P , MA can move along curve ST towards T . Decrement counter C by one. Continue moving along the obstacle boundary (Refer to example in Fig. 5b). Go to step 3.
- iii. At P , MA cannot move along curve ST towards T . Increment counter C by one. Continue moving along the obstacle boundary (Refer to example in Fig. 5c). Go to step 3.

6 Examples

Algorithm *Curv1* is applied to different types of scenes. The curve ST is shown as a continuous line with the arrows showing the direction from S to T . The path generated by algorithm *Curv1* is shown as a dotted-line, with the arrows denoting the direction of MA 's travel. The numbers 1, 2,

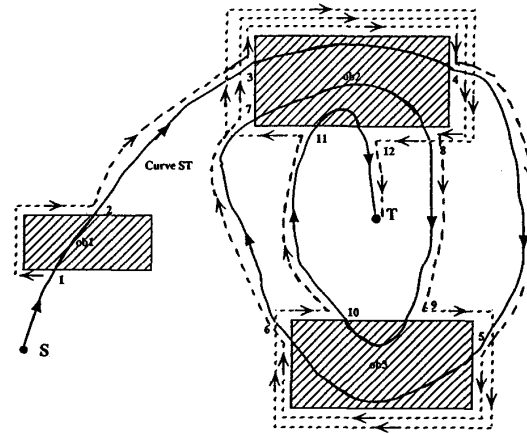


Figure 7: *Curv1* on a simple scene and a complex curve ST

3, ..., denote the sequence of points at which MA meets and leaves the obstacles.

In Fig. 6, the obstacles and curve ST are relatively simple. The path generated for this scene is simple and agrees with the common-sense expectation. In Fig. 7, the obstacles are simple rectangular regions, but the curve ST is a complex spiral. The algorithm produces a complex path, which goes three times over a large section of the obstacle $ob2$.

A complex maze-like scene is shown in Fig. 8, with a relatively simple curve ST . For such a complex scene, a common sense approach is more likely to fail. Algorithm *Curv1* has performed reasonably well for this scene by generating a path that visits only some sections of the obstacle boundaries twice.

7 Characteristics of Algorithm Curv1

Now we shall discuss the various characteristics of the algorithm *Curv1*. It is essential to establish that *Curv1* indeed takes MA from S to target T , without getting into any infinite loop. This is equivalent to showing that there exists a finite upper bound on the length of the path generated.

If the algorithm *Curv1* puts MA into an infinite loop, following are the only two possible ways in which that can happen.

Definition 2 In a *Type1* loop, MA keeps going around an obstacle boundary without ever leaving it.

In a *Type2* loop, MA leaves an obstacle at some leave point more than once.

A *Type1* loop consists entirely of an obstacle boundary, while a *Type2* loop consists of sections of obstacle boundaries and curve ST .

Before proceeding to prove that these two types of infinite loops are not possible under algorithm *Curv1*, it is necessary to discuss the issue of target reachability.

The Reachability of Target T : The target T is reachable if there exists a simple curve in the plane that can join S and T , without entering any obstacle. The target T becomes unreachable when either S or T is trapped

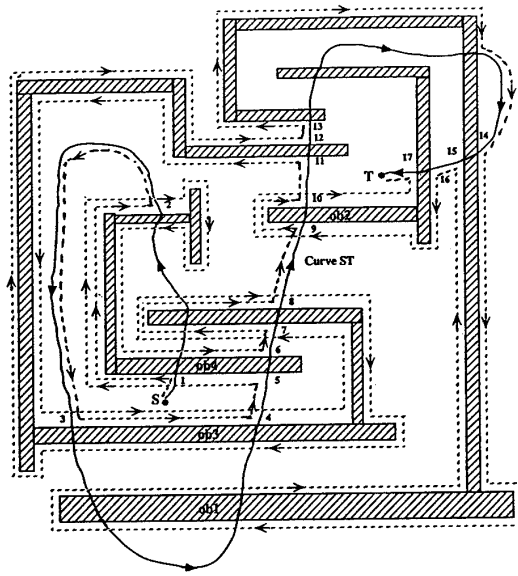


Figure 8: Algorithm *Curv1* applied to a maze-like scene

inside an obstacle. The metric algorithms for problem *P0* [3, 5] can determine, within a finite length of travel, whether target *T* can be reached or not.

The non-metric *Pledge* algorithm cannot determine if the maze it is in has indeed an opening to escape. So, in a closed maze *MA* will simply go around the maze infinitely many times. The same is true for the new algorithm *Curv1*. This problem arises due to a non-metric algorithm's inability to determine whether a given point has already been visited or not. Hence, for algorithm *Curv1*, it is necessary to assume that the target is always reachable. This is not a serious handicap as effective heuristics can be devised to deal with unreachable targets. The target reachability is assumed for all the following results.

We shall first show that a *Type1* infinite loop cannot occur under algorithm *Curv1*.

Theorem 1 *Under algorithm Curv1, while moving along curve ST, let MA meet an obstacle ob1 at a hit point point H. Then MA cannot return to hit point H before leaving the obstacle ob1 at least once.*

That is, under algorithm Curv1, a Type1 infinite loop cannot occur.

Proof: The proof is in three parts.

When *MA* goes around an obstacle boundary it may meet the curve *ST* at some point, say, *P*. At *P*, if the curve *ST* is directed into the obstacle (refer to Fig. 5c), then *P* called an *inward crossing point*. If the curve *ST* is directed out of the obstacle (refer to Fig. 5b), then *P* is called an *outward crossing point*. All other points on an obstacle boundary are called *non-crossing points*.

Lemma 1 *Starting from a non-crossing point, if MA makes a trip exactly once around an obstacle boundary, it will meet the same number of inward and outward crossing points.*

Proof: Assuming the target reachability (i.e. both *S* and *T* are not trapped inside any obstacle), this result follows directly from the Jordon Curve Theorem [8].

Lemma 2 *Under algorithm Curv1, the counter C is always either zero or positive (i.e. it can't be negative).*

Proof: Under algorithm *Curv1* the counter *C* reads zero from the instant *MA* leaves an obstacle boundary to follow curve *ST* to the instant it meets an obstacle. Hence, we need to only check if the counter can be negative when *MA* follows an obstacle boundary.

As per the algorithm, the counter can only change in steps of ± 1 . Hence, to read any negative integer it must first change from zero to -1 . That is, *MA* must meet an outward crossing point with the counter reading zero. But, this is not possible, as under such a situation, the counter must remain at zero and *MA* must leave the obstacle boundary. This proves the lemma.

Lemma 3 *The theorem follows from the lemmas 1 and 2.*

Proof: The proof is by contradiction.

Let us assume that *MA* indeed returns to the hit point *H* without ever leaving the obstacle *ob1*.

According to Lemma 1, starting from a non-crossing point, *MA* meets the same number of inward and outward crossing points. Then, starting from an inward crossing point *H* and not counting it (according to the algorithm), *MA* shall meet one extra outward crossing point before returning to *H*.

That means, the counter *C* must read -1 , just before *MA* returns to *H*. But, according to Lemma 2 the counter *C* cannot have a negative reading. This contradiction implies that the assumption of *MA* not leaving the obstacle boundary and returning to *H* cannot be true. This proves the theorem.

Next we prove that a *Type2* infinite loop is also not possible.

Theorem 2 *Under algorithm Curv1, MA cannot leave an obstacle boundary from any leave point (i.e. outward crossing point) more than once.*

That is, under algorithm Curv1, a Type2 infinite loop cannot occur.

Proof: Some new terminology is introduced first. Refer to Fig. 9 for the following. H_i denotes the i_{th} hit point on *MA*'s path. For example, after starting from *S* and moving along curve *ST*, *MA* meets the first obstacle at H_1 . Similarly, L_i denotes the i_{th} time *MA* leaves an obstacle boundary. For example, after meeting the first obstacle at H_1 , *MA* leaves the same at L_1 . That is, the hit point H_i and the leave point L_i lie on the same obstacle boundary, and H_i is always followed by L_i . The consecutive hit or leave points (e.g. H_i and H_{i+1}) may or may not lie on the same obstacle boundary.

To tie up with the terminology of crossing points, each leave point (hit point) is an outward (inward) crossing point. Also, since theorem is not proved yet, the same outward (inward) point may be defined as more than one leave (hit) point (e.g. an outward point *P* is L_i, L_k , etc.).

First the following result is obtained.

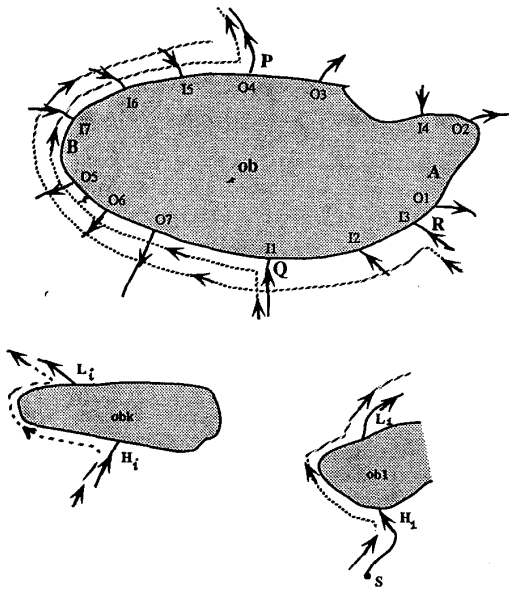


Figure 9: The hit and leave points in the path generated by Curv1

Lemma 4 Let MA leave an obstacle ob at an outward crossing point P , after reaching the obstacle ob through an inward crossing point Q . Then, MA can again leave from the outward crossing point P , if and only if, it reaches obstacle ob through the same inward crossing point Q .

That is, there exists a unique pairing between the inward and outward crossing points, such that, in order to leave the obstacle boundary at one, MA must reach the obstacle through its pair.

Proof: Refer to Fig. 9 for the following. Let MA reach obstacle ob the second time through another inward crossing point R .

The point R can be in one of the two intervals (on the obstacle boundary), PAQ and PBQ (Refer to Fig. 9).

First, let R be in the interval PAQ . After meeting the obstacle, let MA move along the boundary to the left, cross the point Q and reach P . To be able leave at P the second time, the counter C must read zero (as in the first time) when MA reaches P .

During the first trip, C must have been reading zero when MA started following the obstacle boundary at Q . To be able to leave at P again, the counter must read zero after MA crosses Q (moving from R) the second time. Since, Q is an inward crossing point, this is possible only if counter C reads -1 when MA meets Q the second time. Lemma 2 of Theorem 1 showed that C can never read negative. Hence, it is not possible for MA to leave at P again after entering at R . The case of R being in the interval PBQ is also proved along the same lines. This proves the lemma.

We now use the result of the lemma to prove the theorem. The proof is by contradiction.

Let us assume that MA can leave an obstacle the second time at a leave point L_i . Then, from the above lemma it

follows that MA must have reached the obstacle boundary the second time at the hit point H_i . To do so, MA must have left by leave point L_{i-1} the second time, to reach H_i . Continuing this logic, MA must have reached an obstacle again at the first hit point H_1 . This is impossible, as this implies that MA has started again at S (MA always moves along curve ST towards T , and hence can never go back to S).

This is a contradiction. Hence the assumption that MA can leave at a leave point the second time cannot be true.

This proves the theorem.

Corollary 1 Under algorithm Curv1, MA travels along any segment of curve ST at most once.

Proof: Follows from Theorem 2.

Using the result that infinite loops are not possible under Curv1, an upper bound for the path length is derived next.

Theorem 3 The path length L generated under the algorithm Curv1, when MA moves from S to T along a curve ST , cannot exceed the limit,

$$L \leq \sum n_i p_i / 2 + D, \quad (1)$$

where p_i is perimeter of the i_{th} obstacle met by MA and n_i is the number of intersections the curve ST makes with the i_{th} obstacle. D is the length of the curve ST .

Proof: Let n_i be the total number of crossing points (with curve ST) on the obstacle ob_i 's boundary. There are an equal number, $n_i/2$, of inward and outward crossing points (Lemma 1 of Theorem 1). An example of the segments of curve ST at these inward (denoted as I_1, I_2 , etc.) and outward (denoted as O_1, O_2 , etc.) crossing points is shown for obstacle ob in Fig. 9.

MA can reach ob_i 's boundary only by traveling along one of those segments of curve ST that end at an inward crossing point. Similarly MA can leave obstacle ob_i only along one of those curve segments that start at an outward crossing point.

Corollary 1 of Theorem 2 states that MA can travel along any of these curve segments only once. That is, during every visit, MA reaches ob_i through a so far unvisited curve segment, and leaves it through another so far unvisited curve segment. That means, MA can visit or leave obstacle ob_i only at most $n_i/2$ times. Let us assume the worst case possibility that, for obstacle ob_i , MA indeed does this.

Theorem 1 ensures that, during each of these visits to ob_i , MA can go around the obstacle boundary at most only once (actually, less than once). That means, after all its $n_i/2$ visits, MA could have traveled only at most $n_i/2$ times along any interval IN on ob_i 's boundary.

In the worst case, this interval IN can be almost the entire perimeter p_i of the obstacle ob_i . That means, the worst case path length along the obstacle boundary is $n_i p_i / 2$.

Since MA travels only once along any segment of curve ST (Corollary 1 of Theorem 2), the worst case path length along curve ST is, its length D .

The total worst case path length, considering the obstacle ob_i alone, is then, $n_i p_i / 2 + D$. Since, the same argument holds for any other obstacle, the theorem follows.

8 The Relation between Pledge Algorithm and Curv1

Section 4 discusses the general non-applicability of the *Pledge* algorithm to curve-following. The *Pledge* algorithm can be applied as such only when the curve *ST* is a straight line. The new algorithm *Curv1* works for any general curve *ST*.

Fig. 3 shows the path generated by *Pledge* algorithm for a straight curve *ST*. It can be easily verified that algorithm *Curv1* also produces the same path for that scene. The following lemma states that this true for any scene.

Lemma 5 *The algorithms Pledge and Curv1 produce the same path for any scene when the curve ST is a straight line.*

This result implies that the 'angle of turn' criterion of the *Pledge* algorithm is equivalent to the 'counting the crossing points' criterion of algorithm *Curv1*. But, the equivalence holds good only for a straight *ST* and breaks down when a general curve *ST* is considered. That is, for a general curve, the 'counting the crossing points criterion' continues to remain valid while the 'angle of turn criterion' becomes ill-defined. Hence, in the context of curve-following, the algorithm *Curv1* can be considered as a generalization of the *Pledge* algorithm.

The equivalence of *Curv1* with the *Pledge* algorithm for a straight guide-track also means that the generality of the algorithm *Curv1* does not result in unduly complicated paths for simple scenes.

But, it must also be noted that the *Pledge* algorithm is more a maze-searching procedure than a target seeking one. It achieves maze-searching without any guiding curve while algorithm *Curv1* cannot work without a guiding curve. In this respect, *Pledge* algorithm is a distinct method suited for a different goal, as compared to algorithm *Curv1*.

There is another deeper connection between these two algorithms. The algorithm *Curv1* is based on the fact that the curve *ST* intersects any obstacle boundary at an even number of points. This fact follows from the Jordan Curve Theorem [8]. The basic idea of the *Pledge* algorithm is that the total angle of turn around a simple closed planar curve is ± 360 deg. This fact also follows from the Jordan Curve Theorem. Thus, it is interesting to note that different interpretations of the same theorem have given rise to different algorithms, with each performing a certain task that the other cannot.

9 Conclusions

The new, non-heuristic algorithm *Curv1* performs the following of a given curve *ST* in an unknown environment, without using position or distance information, or any physical mark. The curve-following is achieved through counting the number of times the robot crosses the directed curve *ST*.

Algorithm *Curv1* is a generalization of the *Pledge* algorithm, for the curve following task. These two algorithms are based on different, but equivalent, interpretations of the Jordan Curve Theorem. Interestingly, the two algorithms turn out to be distinct in that each can perform a certain unique task that the other cannot.

In a practical autonomous robot navigation, determining the robot's location is generally a non-trivial task requiring elaborate land marks and detection systems. The

algorithm *Curv1* does not need any position or distance information, and therefore can be quite practical.

Under *Curv1*, *MA* can visit a portion of an obstacle boundary up to $n/2$ times, where n is the number of intersections the curve *ST* makes with that obstacle. It may be possible to reduce these multiple repeat trips and achieve a smaller upper bound on the the path length.

Acknowledgements

The authors wish to thank Dr. Hashimoto for his encouragement and guidance.

References

- [1] *Algorithmic Motion Planning*, C. K. Yap, Advances in Robotics, Vol. 1: Algorithmic and Geometric Aspects (J. T. Schwartz and C. K. Yap Eds.), 1987.
- [2] *Real-Time Obstacle Avoidance for Manipulators and Mobile Robots*, O. Khatib, The International Journal of Robotics Research, Vol. 5, No. 1, 1986.
- [3] *Path Planning Strategies For A Point Mobile Automaton Moving Amidst Unknown Obstacles Of Arbitrary Shape*, V. J. Lumelsky, A. A. Stepanov, Algorithmica, Springer-Verlag, 1987.
- [4] *A Comparative Study On The Path Length Performance Of Maze-Searching And Robot Motion Planning Algorithms*, V. J. Lumelsky, IEEE Transactions On Robotics And Automation, Vol. 7, No. 1, 1991.
- [5] *Path Planning For Moving A Point Object Amidst Unknown Obstacles In A Plane: A New Algorithm And A General Theory For Algorithm Development*, A. Sankaranarayanan And M. Vidyasagar, Proc. of the 29th IEEE Conference on Decision and Control, 1990.
- [6] *Survey Of Automated Guided Vehicle In Japanese Factory*, Toshihiro Tsumura, IEEE Conference on Robotics and Automation, 1986.
- [7] *The Turtle Geometry*, Abelson, H. and DiSessa, E., MIT Press, Cambridge, 1980, pp. 176-199.
- [8] *What is Mathematics?*, Richard Courant and Herbert Robbins, Oxford University Press, 1941.